Cascade Prompts Playbook

How to Use This Document

This document contains two meticulously crafted prompts designed for Cascade (Windsurf's AI coding assistant). Together, they represent the most efficient path to autonomously building and then hardening the InsightHub application.

- **Prompt 1** — "The Builder" — builds the entire application from scratch in a fresh workspace
- **Prompt 2** — "The Auditor" — performs deep review, bug hunting, and quality hardening on the built app

---

Prompt 1: The Builder

> ***Purpose:*** *Hand this to a fresh Cascade instance in an empty directory. It should be able to build 80-90% of InsightHub autonomously, including scaffolding, data model, AI integration, UI, and deployment infrastructure.*

```
# BUILD: InsightHub — AI-Powered Dashboard Builder (Full Autonomous Build)

You are building "InsightHub" — a production-grade, internal self-service BI platform where employees
build, customize, and share data dashboards using natural language. An AI assistant (Claude) generates
dashboards in real-time via chat.

**Live reference:** https://dashboards.jeffcoy.net

---

## TECH STACK (Non-Negotiable)

| Layer | Technology | Version |
|-------|-----------|---------|
| Framework | Next.js (App Router) | 16.x |
| Language | TypeScript | strict mode |
| Styling | Tailwind CSS | v4 |
| UI Components | Radix UI primitives | latest |
| Charts | Recharts | v3.x |
| AI | Anthropic Claude SDK | @anthropic-ai/sdk ^0.90 |
| Database | SQLite via Prisma ORM | Prisma 5.x |
| State | Zustand | v5 |
| Auth | NextAuth.js v4 | Google OAuth (bypass in dev) |
| Icons | Lucide React | latest |
| Testing | Playwright | E2E |
| Validation | Zod | v4 |

**CRITICAL:** This uses Next.js 16 with breaking changes. Before writing any code, check
`node_modules/next/dist/docs/` for current API conventions. Do NOT assume Next.js 14/15 patterns work.

---

## ARCHITECTURE OVERVIEW
```

User types message → ChatPanel → POST /api/chat (SSE stream) → Claude Sonnet 4 ↓ Schema patches (JSON) ↓ Canvas re-renders ← Zustand store ← Schema patcher applies patches

```
- **DashboardSchema** is a JSON blob stored in `DashboardVersion.schema`
- AI returns **patches** (`add_widget`, `update_widget`, `remove_widget`, `use_widget`), never full schemas
- Widget library templates are injected into the system prompt so Claude knows available patterns
- Glossary terms are injected so Claude uses correct business terminology
- A 3-layer Data Integrity Verification pipeline validates AI output

---

## DATABASE SCHEMA (Prisma + SQLite)

```prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id          String           @id @default(cuid())
  email       String           @unique
  name        String
  avatarUrl   String?
  role        String           @default("VIEWER")  // VIEWER, CREATOR, POWER_USER, ADMIN
  department  String?
  hasOnboarded Boolean         @default(false)
  createdAt   DateTime         @default(now())
  lastLoginAt DateTime?
  dashboards  Dashboard[]      @relation("owner")
  sharedWith  DashboardShare[]
  chatSessions ChatSession[]
  auditLogs        AuditLog[]
  publishedWidgets WidgetTemplate[] @relation("publishedWidgets")
  permissionAssignments UserPermissionAssignment[]
}

model Dashboard {
  id             String             @id @default(cuid())
  title          String
  description    String?
  thumbnailUrl   String?
  ownerId        String
  owner          User              @relation("owner", fields: [ownerId], references: [id])
  folderId       String?
  folder         Folder?            @relation(fields: [folderId], references: [id])
  currentVersion Int               @default(1)
```
```

```
  versions       DashboardVersion[]
  shares         DashboardShare[]
  tags           String             @default("")
  isTemplate     Boolean            @default(false)
  isPublic       Boolean            @default(false)
  createdAt      DateTime           @default(now())
  updatedAt      DateTime           @updatedAt
  archivedAt     DateTime?
  @@index([ownerId])
  @@index([folderId])
}

model DashboardVersion {
  id          String    @id @default(cuid())
  dashboardId String
  dashboard   Dashboard @relation(fields: [dashboardId], references: [id], onDelete: Cascade)
  version     Int
  schema      String    // JSON blob: DashboardSchema
  changeNote  String?
  createdAt   DateTime  @default(now())
  createdBy   String
  @@index([dashboardId, version])
}

model DashboardShare {
  id          String    @id @default(cuid())
  dashboardId String
  dashboard   Dashboard @relation(fields: [dashboardId], references: [id], onDelete: Cascade)
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  permission  String    @default("VIEW")  // VIEW, COMMENT, EDIT
  createdAt   DateTime  @default(now())
  @@unique([dashboardId, userId])
}

model Folder {
  id         String     @id @default(cuid())
  name       String
  parentId   String?
  parent     Folder?    @relation("children", fields: [parentId], references: [id])
  children   Folder[]   @relation("children")
  dashboards Dashboard[]
  ownerId    String
  visibility String     @default("PRIVATE")  // PRIVATE, TEAM, PUBLIC
  createdAt  DateTime   @default(now())
}

model ChatSession {
  id          String        @id @default(cuid())
  userId      String
  user        User          @relation(fields: [userId], references: [id])
  dashboardId String?
```

```
  messages    ChatMessage[]
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
}

model ChatMessage {
  id           String      @id @default(cuid())
  sessionId    String
  session      ChatSession @relation(fields: [sessionId], references: [id], onDelete: Cascade)
  role         String      // USER, ASSISTANT, SYSTEM
  content      String
  schemaChange String?     // JSON: the patches this message produced
  createdAt    DateTime    @default(now())
}

model GlossaryTerm {
  id             String   @id @default(cuid())
  term           String   @unique
  definition     String
  formula        String?
  category       String
  examples       String?
  relatedTerms   String   @default("")
  dataSource     String?
  lastReviewedAt DateTime?
  approvedBy     String?
  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
  @@index([category])
}

model WidgetTemplate {
  id                   String    @id @default(cuid())
  title                String
  description          String?
  type                 String
  tags                 String    @default("")
  config               String    // JSON: WidgetConfig
  sourceDashboardId    String?
  sourceDashboardTitle String?
  publishedById        String?
  publishedBy          User?     @relation("publishedWidgets", fields: [publishedById], references: [id])
  usageCount           Int       @default(0)
  isPublic             Boolean   @default(true)
  createdAt            DateTime  @default(now())
  updatedAt            DateTime  @updatedAt
  @@index([type])
  @@index([publishedById])
}

model AuditLog {
  id           String   @id @default(cuid())
```

```
  userId       String
  user         User     @relation(fields: [userId], references: [id])
  action       String
  resourceType String
  resourceId   String
  metadata     String?
  createdAt    DateTime @default(now())
  @@index([userId])
  @@index([resourceType, resourceId])
  @@index([createdAt])
}

// --- Sample Data Tables (Phase 1 Demo) ---

model SampleCustomer {
  id              Int                    @id @default(autoincrement())
  name            String
  email           String
  company         String
  plan            String
  region          String
  signupDate      DateTime               @map("signup_date")
  cancelledDate   DateTime?              @map("cancelled_date")
  monthlyRevenue  Float                  @map("monthly_revenue")
  accountManager  String?                @map("account_manager")
  subscriptions   SampleSubscription[]
  tickets         SampleTicket[]
  revenue         SampleRevenue[]
  usage           SampleUsage[]
  @@map("sample_customers")
}

model SampleSubscription {
  id            Int             @id @default(autoincrement())
  customerId    Int             @map("customer_id")
  customer      SampleCustomer  @relation(fields: [customerId], references: [id], onDelete: Cascade)
  plan          String
  status        String
  startDate     DateTime        @map("start_date")
  endDate       DateTime?       @map("end_date")
  monthlyAmount Float           @map("monthly_amount")
  addOns        String?         @map("add_ons")
  @@map("sample_subscriptions")
}

model SampleTicket {
  id                      Int             @id @default(autoincrement())
  customerId              Int             @map("customer_id")
  customer                SampleCustomer  @relation(fields: [customerId], references: [id], onDelete:
Cascade)
  subject                 String
  category                String
```

```
  priority                String
  status                  String
  channel                 String
  createdAt               DateTime        @map("created_at")
  resolvedAt              DateTime?       @map("resolved_at")
  firstResponseMinutes    Int?            @map("first_response_minutes")
  satisfactionScore       Int?            @map("satisfaction_score")
  agent                   String?
  team                    String?
  @@map("sample_tickets")
}

model SampleRevenue {
  id         Int             @id @default(autoincrement())
  customerId Int             @map("customer_id")
  customer   SampleCustomer  @relation(fields: [customerId], references: [id], onDelete: Cascade)
  eventType  String          @map("event_type")
  amount     Float
  eventDate  DateTime        @map("event_date")
  planFrom   String?         @map("plan_from")
  planTo     String?         @map("plan_to")
  @@map("sample_revenue")
}

model SampleUsage {
  id         Int             @id @default(autoincrement())
  customerId Int             @map("customer_id")
  customer   SampleCustomer  @relation(fields: [customerId], references: [id], onDelete: Cascade)
  feature    String
  usageCount Int             @map("usage_count")
  usageDate  DateTime        @map("usage_date")
  @@map("sample_usage")
}

model SampleDeal {
  id          Int       @id @default(autoincrement())
  company     String
  contact     String
  stage       String
  amount      Float
  probability Int
  source      String
  region      String
  createdAt   DateTime  @map("created_at")
  closedAt    DateTime? @map("closed_at")
  owner       String
  @@map("sample_deals")
}

// --- Permission System ---

model PermissionGroup {
```

```
  id                 String                     @id @default(cuid())
  name               String                     @unique
  description        String?
  isSystem           Boolean                    @default(false)
  featurePermissions String                     @default("{}")
  dataPermissions    String                     @default("{}")
  createdAt          DateTime                   @default(now())
  updatedAt          DateTime                   @updatedAt
  userAssignments    UserPermissionAssignment[]
  dataAccessRules    DataAccessRule[]
  @@index([isSystem])
}

model UserPermissionAssignment {
  id                 String           @id @default(cuid())
  userId             String
  user               User             @relation(fields: [userId], references: [id], onDelete: Cascade)
  permissionGroupId  String
  permissionGroup    PermissionGroup  @relation(fields: [permissionGroupId], references: [id], onDelete:
Cascade)
  customOverrides    String           @default("{}")
  assignedBy         String
  assignedAt         DateTime         @default(now())
  @@unique([userId, permissionGroupId])
  @@index([userId])
  @@index([permissionGroupId])
}

model DataAccessRule {
  id                 String           @id @default(cuid())
  permissionGroupId  String
  permissionGroup    PermissionGroup  @relation(fields: [permissionGroupId], references: [id], onDelete:
Cascade)
  dataCategory       String
  accessLevel        String           @default("NONE")
  filterCriteria     String?
  metricAccessRules  MetricAccessRule[]
  createdAt          DateTime         @default(now())
  updatedAt          DateTime         @updatedAt
  @@unique([permissionGroupId, dataCategory])
  @@index([permissionGroupId])
  @@index([dataCategory])
}

model MetricAccessRule {
  id                 String         @id @default(cuid())
  dataAccessRuleId  String
  dataAccessRule    DataAccessRule @relation(fields: [dataAccessRuleId], references: [id], onDelete:
Cascade)
  metricName        String
  accessLevel       String         @default("NONE")
  filterCriteria    String?
```

```
  deniedReason      String?
  createdAt         DateTime       @default(now())
  updatedAt         DateTime       @updatedAt
  @@unique([dataAccessRuleId, metricName])
  @@index([dataAccessRuleId])
  @@index([metricName])
}

model DataMetaCategory {
  id                String   @id @default(cuid())
  name              String   @unique
  description       String?
  includedCategories String
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt
  @@index([name])
}
```

CORE TYPES (src/types/index.ts)

```
export interface DashboardSchema {
  layout: { columns: number; rowHeight: number; gap: number };
  globalFilters: FilterConfig[];
  widgets: WidgetConfig[];
}

export interface WidgetConfig {
  id: string;
  type: WidgetType;
  title: string;
  subtitle?: string;
  position: { x: number; y: number; w: number; h: number };
  dataConfig: {
    source: string;
    query?: string;
    filters?: FilterConfig[];
    aggregation?: AggregationConfig;
    groupBy?: string[];
    orderBy?: { field: string; direction: 'asc' | 'desc' }[];
    limit?: number;
  };
  visualConfig: {
    chartType?: ChartType;
    colorScheme?: string;
    showLegend?: boolean;
    showGrid?: boolean;
    showLabels?: boolean;
    stacked?: boolean;
    animate?: boolean;
    thresholds?: ThresholdConfig[];
    customStyles?: Record<string, string>;
```

```
  };
  glossaryTermIds?: string[];
}

export type WidgetType =
  | 'kpi_card' | 'line_chart' | 'bar_chart' | 'area_chart'
  | 'pie_chart' | 'donut_chart' | 'stacked_bar' | 'scatter_plot'
  | 'heatmap' | 'table' | 'pivot_table' | 'funnel'
  | 'gauge' | 'metric_row' | 'text_block' | 'image' | 'divider' | 'map';

export type ChartType =
  | 'line' | 'bar' | 'area' | 'pie' | 'donut' | 'scatter'
  | 'heatmap' | 'funnel' | 'gauge' | 'radar' | 'treemap';

export interface SchemaPatch {
  type: 'add_widget' | 'remove_widget' | 'update_widget' | 'update_layout' | 'update_filters' |
'replace_all' | 'use_widget';
  widgetId?: string;
  widget?: WidgetConfig;
  widgetTemplateId?: string;
  changes?: Partial<WidgetConfig>;
  layout?: Partial<DashboardSchema['layout']>;
  filters?: FilterConfig[];
  schema?: DashboardSchema;
}
```

PROJECT STRUCTURE

```
src/
├── app/
│   ├── layout.tsx              # Root layout (providers, Inter font, theme)
│   ├── page.tsx                # Gallery home (tabbed: My, Shared, Company, Templates)
│   ├── gallery-client.tsx      # Client gallery component with search/filter/folders
│   ├── globals.css             # Tailwind + glassmorphism CSS variables
│   ├── login/page.tsx
│   ├── onboarding/page.tsx
│   ├── dashboard/
│   │   ├── [id]/page.tsx       # Dashboard editor (canvas + chat panel)
│   │   └── new/page.tsx        # New blank dashboard
│   ├── glossary/page.tsx
│   ├── templates/page.tsx
│   ├── admin/                  # Users, Settings, Audit, Permissions, Prompts
│   │   ├── page.tsx
│   │   ├── audit/page.tsx
│   │   ├── settings/page.tsx
│   │   └── permissions/page.tsx
│   ├── about/page.tsx
│   └── api/
│       ├── chat/route.ts       # AI chat endpoint (SSE streaming)
│       ├── dashboards/         # CRUD + versions + share + duplicate
│       ├── data/               # Sample data query engine
```

```
│       ├── glossary/           # Terms CRUD + search
│       ├── admin/              # User management, audit log
│       ├── health/route.ts     # Health check (DB, memory, uptime)
│       ├── widgets/            # Widget library: search, publish, fork
│       ├── folders/            # Folder CRUD
│       ├── voice/route.ts      # OpenAI Whisper transcription
│       └── thumbnails/route.ts # Dashboard thumbnail generation
├── components/
│   ├── chat/
│   │   ├── ChatPanel.tsx       # Persistent side panel with SSE streaming
│   │   └── VoiceWaveform.tsx   # Speech-to-text visual indicator
│   ├── dashboard/
│   │   ├── DashboardCanvas.tsx # 12-column grid renderer
│   │   ├── DashboardToolbar.tsx
│   │   ├── WidgetRenderer.tsx  # Dynamic widget type dispatch
│   │   ├── WidgetConfigPanel.tsx
│   │   ├── ContextMenu.tsx
│   │   └── DragHandle.tsx
│   ├── widgets/
│   │   ├── KpiCard.tsx, LineChartWidget.tsx, BarChartWidget.tsx
│   │   ├── AreaChartWidget.tsx, PieChartWidget.tsx, DataTableWidget.tsx
│   │   ├── GaugeWidget.tsx, FunnelWidget.tsx, TextBlockWidget.tsx
│   │   ├── HeatmapWidget.tsx, ScatterPlotWidget.tsx, PivotTableWidget.tsx
│   │   ├── MetricRowWidget.tsx, WidgetLibraryPanel.tsx
│   │   └── index.ts            # Widget registry
│   ├── gallery/                # Dashboard cards, search, folders
│   ├── glossary/               # Term browser + editor
│   ├── layout/                 # Navbar, ThemeToggle, CommandPalette, MobileNotice
│   ├── versioning/             # Version timeline
│   ├── onboarding/             # Welcome walkthrough
│   └── ui/                     # Toast, Kbd, Tooltip, shared primitives
├── hooks/
│   ├── useAutoSave.ts          # 30s inactivity auto-save
│   ├── useAutoSaveWithThumbnails.ts
│   ├── useKeyboardShortcuts.ts # ⌘K, ⌘Z, ⌘S, etc.
│   ├── useSpeechToText.ts      # Whisper voice input
│   ├── useViewport.ts          # Responsive detection
│   └── useDataSourcePermissions.ts
├── lib/
│   ├── ai/
│   │   ├── prompts.ts          # System prompt builder (injects glossary, schema, permissions)
│   │   ├── schema-patcher.ts   # Apply AI patches to DashboardSchema
│   │   ├── change-summarizer.ts # AI-generated version change notes
│   │   ├── auto-layout.ts     # Intelligent widget positioning
│   │   ├── verify-integrity.ts # 3-layer verification pipeline
│   │   ├── source-field-registry.ts # Valid sources + fields for verification
│   │   └── prompt-overrides.ts # Admin-configurable prompt customization
│   ├── auth/
│   │   ├── config.ts           # NextAuth configuration
│   │   ├── session.ts          # getCurrentUser, canAccessSensitiveData
│   │   └── permissions.ts      # RBAC: category-level + metric-level
│   ├── data/
```

```
│   │   ├── sample-data.ts      # Query engine for sample tables
│   │   ├── widget-library.ts   # Pre-built widget templates
│   │   └── templates.ts        # Dashboard templates (Executive, Support, etc.)
│   ├── db/prisma.ts            # Prisma client singleton
│   ├── env.ts                  # Zod-validated environment variables
│   ├── settings.ts             # System settings (AI model, feature flags)
│   ├── rate-limiter.ts         # Sliding window rate limiter
│   ├── audit.ts                # Audit logging helpers
│   ├── logger.ts               # Structured logging
│   ├── export-utils.ts         # PNG, SVG, CSV export
│   ├── thumbnail-generator.ts  # html-to-image based thumbnails
│   └── utils.ts                # cn() helper, misc
├── stores/
│   └── dashboard-store.ts      # Zustand + undo/redo (50-entry capped history)
└── types/
    └── index.ts
```

CRITICAL IMPLEMENTATION DETAILS

1. AI Chat System (THE CORE FEATURE)

The chat endpoint (`/api/chat`) must:

- Accept: message, currentSchema, conversationHistory, sessionId, dashboardId, stream flag
- Validate input with Zod
- Load glossary terms from DB (fallback: YAML file)
- Build a system prompt that includes: glossary, current schema, available data sources (filtered by user permissions), widget library templates
- Call Claude Sonnet 4 (claude-sonnet-4-20250514) with SSE streaming
- Parse response to extract schema patches (JSON between ```json fences)
- Emit SSE events: `text` (prose), `patch` (individual schema patches), `done` , `verification`
- After patches, run 3-layer integrity verification (if enabled)
- Persist messages to ChatSession

System Prompt Structure:

```
You are a dashboard builder assistant for InsightHub...
[Rules about using glossary definitions, not inventing metrics]
[Company Glossary: {terms}]
[Current Dashboard Schema: {schema}]
[Available Data Sources: {sources filtered by permissions}]
[Widget Library: {available templates}]
[Output format: explanation + JSON patches]
```

2. Widget Rendering

Each widget type is a React component that:

- Receives `WidgetConfig` + resolved data
- Renders using Recharts (charts) or custom components (KPI cards, tables)
- Supports theme-aware colors
- Has hover tooltips, responsive sizing

- A `WidgetRenderer` component dispatches by `widget.type`

3. Dashboard Store (Zustand)

```
interface DashboardState {
  dashboardId: string | null;
  title: string;
  schema: DashboardSchema;
  isDirty: boolean;
  isSaving: boolean;
  isAiWorking: boolean;
  selectedWidgetId: string | null;
  history: HistoryEntry[];      // Capped at 50
  historyIndex: number;
  canUndo: boolean;
  canRedo: boolean;
  // Actions: initialize, applyPatch, addWidget, removeWidget, updateWidget,
  //          duplicateWidget, moveWidget, resizeWidget, undo, redo, save...
}
```

4. Data Layer

For Phase 1, sample data is queried via Prisma from SQLite tables. The `/api/data` endpoint accepts a source name and returns formatted data that widgets consume. The seed script (`prisma/seed.ts`) generates:

- ~200 customers across 5 regions, 3 plans
- ~2000 support tickets with realistic distributions
- 18 months of revenue events (3-5% monthly churn)
- Product usage with weekday-heavy patterns
- ~100 sales deals in various pipeline stages

Use @faker-js/faker for realistic data generation.

5. Design System

Dark-first with light mode toggle. Glassmorphism aesthetic:

- Background: `#0a0e14` (dark), `#f8fafc` (light)
- Cards: `rgba(22, 27, 34, 0.85)` with `backdrop-blur(12px)` , subtle borders
- Accent colors: Green (#3fb950), Amber (#d29922), Red (#f85149), Blue (#58a6ff), Purple (#bc8cff), Cyan (#39d2c0)
- Font: Inter (system fallback)
- Animations: fadeIn, hover lifts, smooth transitions
- KPI cards: gradient accent bar at top, hero number (2.5rem, weight 800), trend pill

6. Auth (Dev Mode Bypass)

When `NEXT_PUBLIC_DEV_MODE=true` :

- Skip OAuth entirely
- Auto-login as admin user
- `getCurrentUser()` returns a mock admin

Production uses Google OAuth with `uszoom.com` domain restriction.

7. Security Middleware

- Security headers: X-Frame-Options, CSP, HSTS, X-Content-Type-Options
- Rate limiting on chat endpoint (sliding window)
- Input validation with Zod on all API routes
- Audit logging for all mutations

8. Keyboard Shortcuts & Command Palette

- ⌘K: Command palette (search dashboards, run actions)
- ⌘Z / ⌘⇧Z: Undo/redo
- ⌘S: Save
- /: Focus chat
- ?: Show keyboard shortcuts overlay
- Arrow keys: Nudge selected widget

9. Glossary System

Canonical source: `glossary/terms.yaml` (YAML format) Synced to DB via seed script. Categories: Retention, Revenue, Support, Sales, Product. Each term has: term, definition, formula, category, data_source, related_terms. The AI system prompt includes all glossary terms so it uses correct business definitions.

SAMPLE GLOSSARY (glossary/terms.yaml)

```yaml
- term: "Churn Rate"
  category: "Retention"
  definition: >
    The percentage of customers who cancel their subscription within a given
    period, divided by the total active customers at the start of that period.
    Excludes trial accounts (< 14 days) and internal test accounts.
  formula: "(cancelled_in_period / active_at_period_start) * 100"
  data_source: "sample_subscriptions"
  related_terms: ["MRR", "LTV", "Retention Rate", "NRR"]

- term: "Monthly Recurring Revenue (MRR)"
  category: "Revenue"
  definition: >
    The sum of all active subscription fees normalized to a monthly amount.
    Includes base subscription + add-ons. Excludes one-time fees.
  formula: "SUM(monthly_amount) WHERE status = 'active'"
  data_source: "sample_subscriptions"
  related_terms: ["ARR", "Churn Rate", "NRR"]

- term: "Net Revenue Retention (NRR)"
  category: "Revenue"
  definition: >
    The percentage of recurring revenue retained from existing customers,
    including expansions and contractions but excluding new customers.
  formula: "(MRR_end - MRR_new) / MRR_start * 100"
  related_terms: ["MRR", "Churn Rate"]

- term: "Customer Satisfaction (CSAT)"
```

```yaml
  category: "Support"
  definition: >
    Average satisfaction score from post-resolution surveys. Scale 1-5.
  formula: "AVG(satisfaction_score) WHERE satisfaction_score IS NOT NULL"
  data_source: "sample_tickets"

- term: "First Response Time"
  category: "Support"
  definition: >
    Median time (in minutes) from ticket creation to first agent response.
  formula: "MEDIAN(first_response_minutes)"
  data_source: "sample_tickets"

- term: "Average Revenue Per User (ARPU)"
  category: "Revenue"
  definition: >
    Total MRR divided by total active customers.
  formula: "SUM(monthly_amount WHERE active) / COUNT(DISTINCT active_customers)"

- term: "Lifetime Value (LTV)"
  category: "Revenue"
  definition: >
    Expected total revenue from a customer over their lifetime.
  formula: "ARPU / (Churn Rate / 100)"
  related_terms: ["ARPU", "Churn Rate"]
```

Include 15-20 terms total spanning all categories.

ENVIRONMENT VARIABLES (.env.example)

```
DATABASE_URL="file:./dev.db"
GOOGLE_CLIENT_ID=""
GOOGLE_CLIENT_SECRET=""
NEXTAUTH_SECRET="change-me-in-production"
NEXTAUTH_URL="http://localhost:3000"
ALLOWED_DOMAIN="uszoom.com"
ANTHROPIC_API_KEY=""
OPENAI_API_KEY=""
NEXT_PUBLIC_DEV_MODE="true"
VERIFY_INTEGRITY_ENABLED="true"
VERIFY_INTEGRITY_AI_ENABLED="true"
VERIFY_INTEGRITY_CONFIDENCE_THRESHOLD="0.70"
VERIFY_INTEGRITY_TIMEOUT_MS="5000"
```

DEPLOYMENT

- **Target:** EC2 instance via Tailscale SSH
- **Domain:** dashboards.jeffcoy.net
- **Process:** systemd service running `next start`
- **Reverse proxy:** Nginx with SSL (Let's Encrypt)

- **Deploy script:** `deploy.sh` — rsync code, npm ci, prisma generate, build, restart service, health check, auto-rollback on failure
- **Backup:** SQLite backup script with encryption

Create `deploy.sh`, `scripts/ec2-deploy.sh`, `scripts/backup-db.sh`, `infra/nginx.conf`, `infra/insighthub.service`.

BUILD ORDER

Execute in this sequence:

1. Scaffold Next.js project, install all dependencies, configure TypeScript/Tailwind/ESLint
2. Create Prisma schema + seed script with realistic data
3. Build types system (src/types/index.ts)
4. Build auth system (NextAuth config + dev bypass + session helpers)
5. Build Zustand store with undo/redo
6. Build AI system (prompts.ts, schema-patcher.ts, /api/chat with SSE)
7. Build all widget components (KPI, Line, Bar, Area, Pie, Table, Gauge, Funnel, Text, etc.)
8. Build dashboard canvas + grid layout + widget renderer
9. Build chat panel UI with streaming display
10. Build gallery page with search, tabs, thumbnails
11. Build admin panel (users, settings, audit log, permissions)
12. Build glossary system (YAML sync, browse UI, widget linking)
13. Build keyboard shortcuts, command palette, theme toggle
14. Build auto-save, thumbnail generation, export (PNG/SVG/CSV)
15. Build deployment infrastructure (deploy.sh, nginx, systemd)
16. Build data integrity verification pipeline
17. Add middleware (security headers, CSP, auth enforcement)
18. Create E2E tests with Playwright
19. Set up CI/CD (GitHub Actions: lint + typecheck + build + test)

QUALITY STANDARDS

- All components must be production-quality, not prototypes
- Error boundaries on every page
- Loading states for all async operations
- Mobile-responsive (show "Desktop recommended" notice on mobile)
- Accessible (ARIA labels, keyboard navigation, focus management)
- No `any` types — strict TypeScript throughout
- All API routes validate input with Zod
- All mutations logged to audit trail
- Rate limiting on AI endpoints
- Graceful error handling with user-friendly messages

```
---

---

# Prompt 2: The Auditor

> **Purpose:** After the app is built and running, hand this to a fresh Cascade instance to perform an
```

exhaustive code review, identify bugs, security issues, performance problems, and architecture concerns. This instance should act as a senior staff engineer doing a thorough production readiness review.

```markdown
# AUDIT: InsightHub — Deep Code Review & Production Hardening

You are performing an exhaustive senior staff-level code review of InsightHub, an AI-powered dashboard builder built with Next.js 16, TypeScript, Prisma/SQLite, Zustand, Recharts, and the Anthropic Claude SDK. The app is live at dashboards.jeffcoy.net.

**Your role:** You are a combined Security Engineer + Performance Engineer + QA Lead + Architecture Reviewer. Your job is to find every bug, vulnerability, performance issue, race condition, and architectural weakness — then fix them.

---

## REVIEW SCOPE (In Priority Order)

### 1. SECURITY AUDIT (Critical)

Examine every file for:

- [ ] **SQL Injection** — Are all Prisma queries parameterized? Any raw SQL?
- [ ] **XSS** — Is user input sanitized before rendering? Check chat messages, dashboard titles, glossary terms, widget text blocks
- [ ] **CSRF** — Is NextAuth's CSRF protection properly configured? Any unprotected mutation endpoints?
- [ ] **Auth Bypass** — Can unauthenticated users access protected routes? Test middleware coverage. Is dev mode properly gated?
- [ ] **IDOR** — Can User A access User B's private dashboards by guessing IDs? Check all ownership validation
- [ ] **Rate Limiting** — Is the AI chat endpoint properly rate-limited? Can someone exhaust the Anthropic API budget?
- [ ] **Input Validation** — Does every API route validate with Zod? Check max lengths, type coercion, missing fields
- [ ] **SSE Injection** — Can a user inject malicious content into the AI SSE stream?
- [ ] **Path Traversal** — Check file operations (thumbnail storage, glossary YAML loading)
- [ ] **Secrets Exposure** — Are API keys or secrets ever logged, sent to client, or in source control?
- [ ] **CSP Compliance** — Is the Content Security Policy correct and not overly permissive?
- [ ] **Permission Escalation** — Can a VIEWER promote themselves to ADMIN? Check all role checks

### 2. BUG HUNTING (High Priority)

Look for:

- [ ] **Race Conditions** — Auto-save + manual save collisions. Multiple SSE streams simultaneously. Undo during AI streaming
- [ ] **State Corruption** — Can the Zustand store reach an invalid state? What happens if schema patches reference non-existent widgets?
- [ ] **Memory Leaks** — Check useEffect cleanup, event listener removal, SSE connection lifecycle, AbortController usage
```

- [ ] **Error Boundaries** — Does every page have error handling? What happens when Claude API is down?
- [ ] **Edge Cases** — Empty dashboards, deleted dashboards still referenced, very long chat messages, special characters in titles
- [ ] **Data Integrity** — Can the verification pipeline produce false positives that confuse users? Is caching correct?
- [ ] **Stale Closures** — React hooks capturing stale state, especially in callbacks passed to streaming handlers
- [ ] **TypeScript Soundness** — Any `as any` casts hiding real type errors? Unsafe assertions?
- [ ] **API Error Handling** — Do all fetch calls handle network errors, 4xx, 5xx gracefully?
- [ ] **Database Constraints** — Can duplicate records be created? Are cascading deletes correct?

3. PERFORMANCE AUDIT

- [ ] **Bundle Size** — Are heavy libraries (Recharts, html-to-image) tree-shaken? Dynamic imports where possible?
- [ ] **Rendering** — Are widgets memoized? Does moving one widget re-render all widgets? Check React.memo, useMemo, useCallback usage
- [ ] **Data Fetching** — N+1 queries? Unnecessary refetches? SWR/cache strategy?
- [ ] **SSE Efficiency** — Does the chat stream properly backpressure? Memory accumulation during long conversations?
- [ ] **Image/Thumbnail** — Are thumbnails optimized? Lazy loaded?
- [ ] **SQLite** — Are indexes correct? Any full table scans on large tables?
- [ ] **Client State** — Is the Zustand store serializing correctly? History cap working?

4. ARCHITECTURE REVIEW

- [ ] **Code Duplication** — Any logic duplicated between client/server that could be shared?
- [ ] **Separation of Concerns** — Are API routes lean (just routing/validation) or do they contain business logic?
- [ ] **Error Consistency** — Uniform error response format across all endpoints?
- [ ] **Type Safety End-to-End** — Are API responses typed on the client? Or just `any`?
- [ ] **Feature Flags** — Are unfinished features (Snowflake, Redis) properly gated?
- [ ] **Dead Code** — Unused imports, unreachable code paths, commented-out blocks
- [ ] **Dependency Health** — Any deprecated packages? Security vulnerabilities in dependencies?

5. UX/ACCESSIBILITY

- [ ] **Keyboard Navigation** — Can every action be performed without a mouse?
- [ ] **Screen Reader** — ARIA labels on interactive elements? Semantic HTML?
- [ ] **Color Contrast** — Do text/background combinations meet WCAG 2.1 AA?
- [ ] **Loading States** — Skeleton screens or spinners for all async content?
- [ ] **Error Messages** — Are error messages user-friendly (not raw stack traces)?
- [ ] **Mobile** — Does the mobile notice actually prevent broken experiences?

METHODOLOGY

For each issue found:

1. **Identify** — File path, line number, exact problem
2. **Severity** — Critical / High / Medium / Low

3. **Impact** — What can go wrong? Who is affected?
4. **Fix** — Implement the fix directly (don't just report — fix it)
5. **Verify** — Explain how to verify the fix works

SPECIFIC FILES TO SCRUTINIZE

These are the highest-risk files — examine them line-by-line:

- `src/app/api/chat/route.ts` — AI endpoint (SSE, streaming, validation)
- `src/lib/ai/prompts.ts` — System prompt construction (injection risk)
- `src/lib/ai/schema-patcher.ts` — Patch application (corruption risk)
- `src/lib/ai/verify-integrity.ts` — Verification pipeline (correctness)
- `src/stores/dashboard-store.ts` — State management (race conditions)
- `src/components/chat/ChatPanel.tsx` — SSE client (memory leaks, state)
- `src/lib/auth/permissions.ts` — RBAC (bypass risk)
- `src/lib/auth/session.ts` — Session handling (auth bypass)
- `middleware.ts` — Security headers + route protection
- `src/app/api/dashboards/[id]/route.ts` — CRUD (IDOR)
- `src/lib/rate-limiter.ts` — Rate limiting (bypass, memory)
- `prisma/seed.ts` — Data generation (correctness)
- `deploy.sh` — Deployment (security, atomicity)

OUTPUT FORMAT

Structure your review as:

Critical Issues (Fix Immediately)
[issues that could cause data loss, security breach, or app crash]

High Priority (Fix Before Next Deploy)
[bugs that affect user experience or data integrity]

Medium Priority (Fix This Sprint)
[performance issues, code quality, missing error handling]

Low Priority (Tech Debt)
[code cleanup, minor UX issues, nice-to-haves]

Architecture Recommendations
[structural improvements for long-term maintainability]

TESTING REQUIREMENTS

After fixing issues, ensure:

1. `npm run check:types` passes with zero errors

```
2. `npm run lint` passes clean
3. `npm run build` completes successfully
4. All existing E2E tests still pass
5. Write NEW regression tests for any critical bug you fix

---

## CONSTRAINTS

- Do NOT refactor for style alone — only change code that has a real bug or security issue
- Do NOT add new features — this is a hardening pass only
- Prefer minimal upstream fixes over downstream workarounds
- Single-line fixes are preferred when sufficient
- Do NOT delete or weaken existing tests
- If you find an issue you cannot fix (needs architectural change), document it clearly with a proposed approach
```

Appendix: Prompt Design Philosophy

Why Prompt 1 Works

1. **Explicit tech stack with versions** — Prevents the AI from choosing incompatible libraries or deprecated versions
2. **Full Prisma schema inline** — The data model is the foundation; providing it verbatim eliminates guesswork
3. **TypeScript interfaces** — Typed contracts between components prevent integration bugs
4. **Directory structure** — Tells the AI exactly where to put every file
5. **Build order** — Prevents dependency issues (can't build widgets before types exist)
6. **Design system with exact hex values** — Produces consistent visual output without iteration
7. **Architecture diagram** — Shows data flow so the AI connects components correctly
8. **"Critical implementation details"** — Deep specifics on the hardest parts (SSE, Zustand, AI prompts)
9. **Quality standards** — Sets the bar explicitly (no `any`, error boundaries, validation)
10. **Glossary sample** — Gives the AI concrete examples of the domain language

Why Prompt 2 Works

1. **Priority-ordered checklist** — Security first, bugs second, performance third
2. **Specific file list** — Points the AI at high-risk code instead of reviewing everything equally
3. **OWASP-aware categories** — SQL injection, XSS, CSRF, IDOR are explicitly called out
4. **"Fix, don't just report"** — Forces actionable output instead of a list of concerns
5. **Methodology section** — Standardizes the format so findings are consistent and actionable
6. **Testing requirements** — Ensures fixes don't break existing functionality
7. **Constraints** — Prevents scope creep (no refactoring for taste, no new features)
8. **Edge case prompting** — Calls out race conditions, stale closures, and memory leaks that AI often misses

---